



GraphQL

What is GraphQL

In a nutshell:

Created by Facebook in 2012

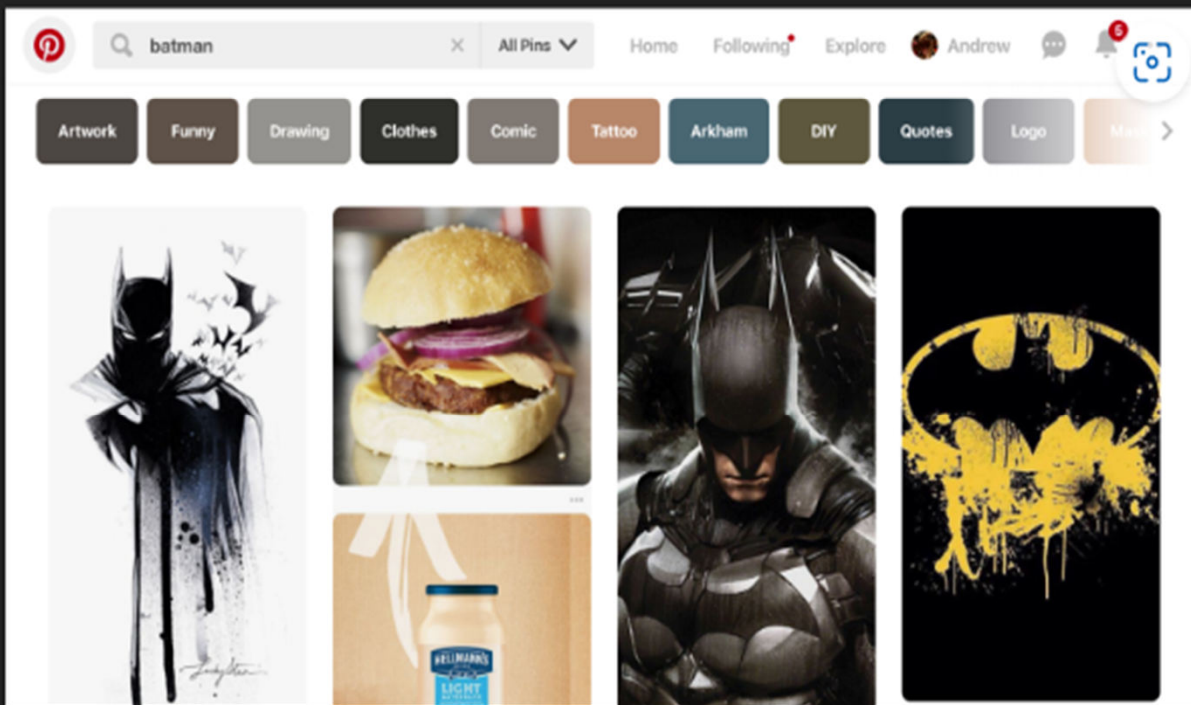
GraphQL is a replacement for **REST**

GraphQL uses a **QUERY LANGUAGE**

GraphQL is a **PATTERN** not a technology

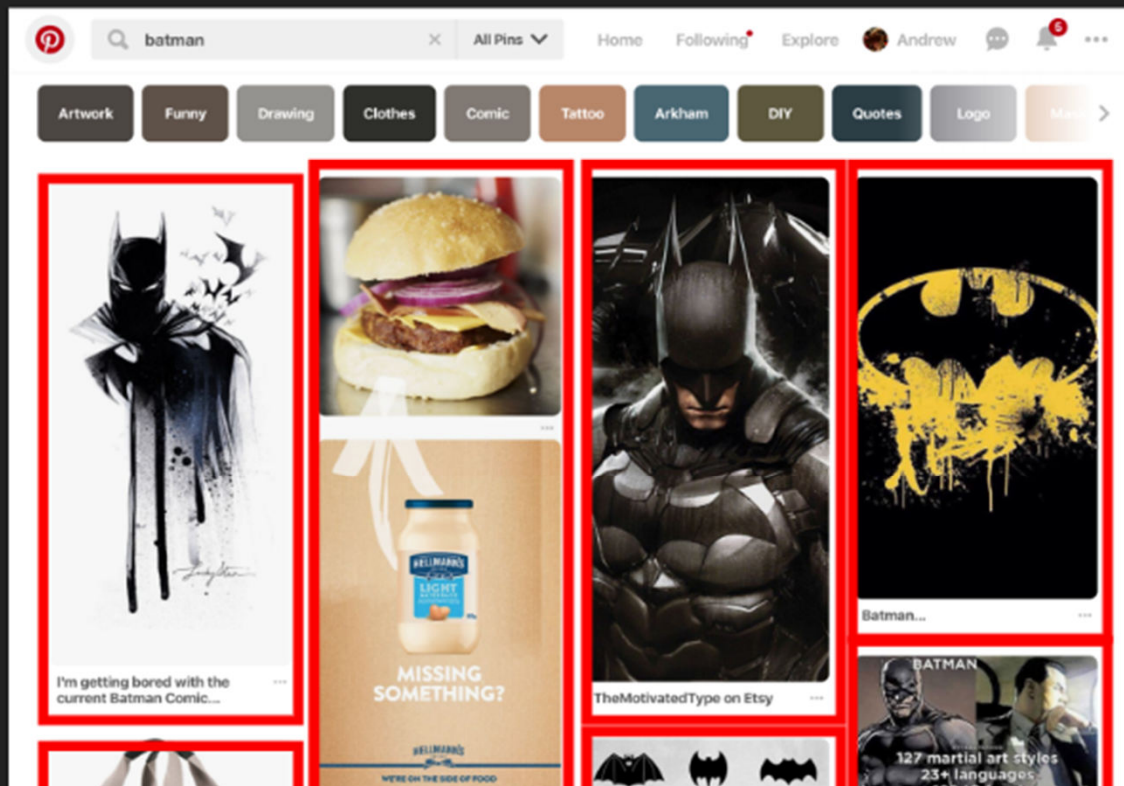
Why choose GraphQL?

REST in applications has a disadvantage with lots of information to display on one screen



Why choose GraphQL?

REST in applications has a disadvantage with lots of information to display on one screen



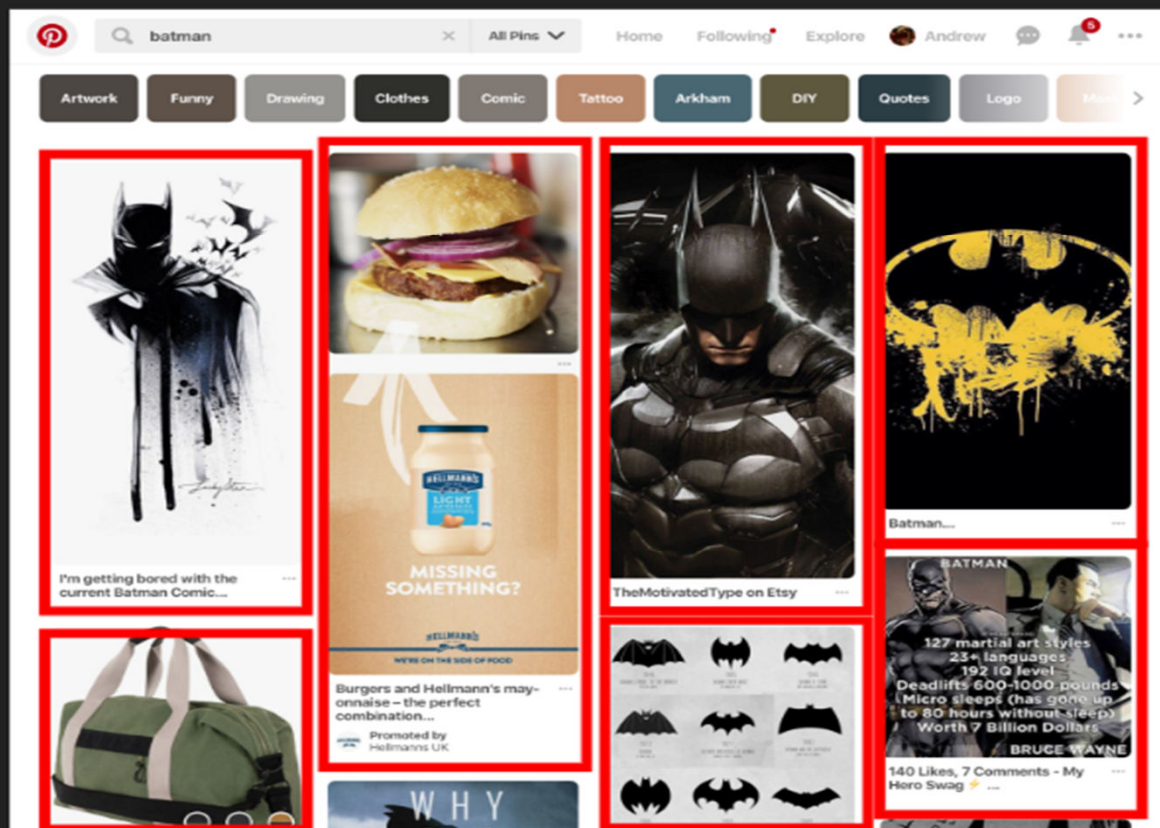
One API Call for collection

Then one call per item for details

```
GET /results
GET /results/1234
GET /results/5678
GET /results/9012
```


GraphQL vs REST

GraphQL can bundle lots of data into one query instead of many



```
POST /graphql

{
  pins(last: 10) {
    imgUrl
    name
    likes
  }
}
```

O = 1

Other benefits

Strongly typed schema

- Can be shared with client & server
- Can validate requests at build time
- Auto complete queries and mutations
- Can be mocked easily

No under-fetching and over-fetching

- Reduce those spinner times!

Passionate community

- This project has been around a while
- And will last a while too

Schema definition

First, you describe your data with a schema

```
type Query {  
  pins: [Pin]  
}  
  
type Pin {  
  id: ID  
  name: String  
  imgUrl: String  
  likes: Number  
}
```

Strongly Typed

Types can be a Scalar
(boolean, string, ID)
or Object with fields

This schema can generate types in Typescript, or Flow

Resolvers

Then you create resolvers for all schema fields in your chosen language using a library that implement the data. Resolvers work at the object property level

```
const types = {
  Query: {
    pins: () => {
      return db.get.pins()
    }
  }
  Pin: (pinData) => {
    return {
      id: pinData._id,
      name: pinData.name.toLowerCase()
      likes: pinData.likes.length
    }
  }
}
```


Queries:

- Retrieve data
- Specify which fields are returned
- Can pass in variables, great for pagination

```
query HeroNameAndFriends($episode: Episode) {  
  hero(episode: $episode) {  
    name  
    friends {  
      name  
    }  
  }  
}
```

Query

```
{  
  "episode": "JEDI"  
}
```

Variables

```
{  
  "data": {  
    "hero": {  
      "name": "R2-D2",  
      "friends": [  
        {  
          "name": "Luke Skywalker"  
        },  
        {  
          "name": "Han Solo"  
        }  
      ]  
    }  
  }  
}
```

Response

Example with NodeJS + Express

```
npm install --save apollo-server-express@1 graphql-tools graphql express body-parser
```



server.js

```
const express = require('express');
const bodyParser = require('body-parser');
const { graphqlExpress } = require('apollo-server-express');
const { makeExecutableSchema } = require('graphql-tools');

const books = [
  {
    title: "Harry Potter and the Sorcerer's stone",
    author: 'J.K. Rowling',
  },
  {
    title: 'Jurassic Park',
    author: 'Michael Crichton',
  },
];

const typeDefs = `
  type Query { books: [Book] }
  type Book { title: String, author: String }
`;

const resolvers = {
  Query: { books: () => books },
};

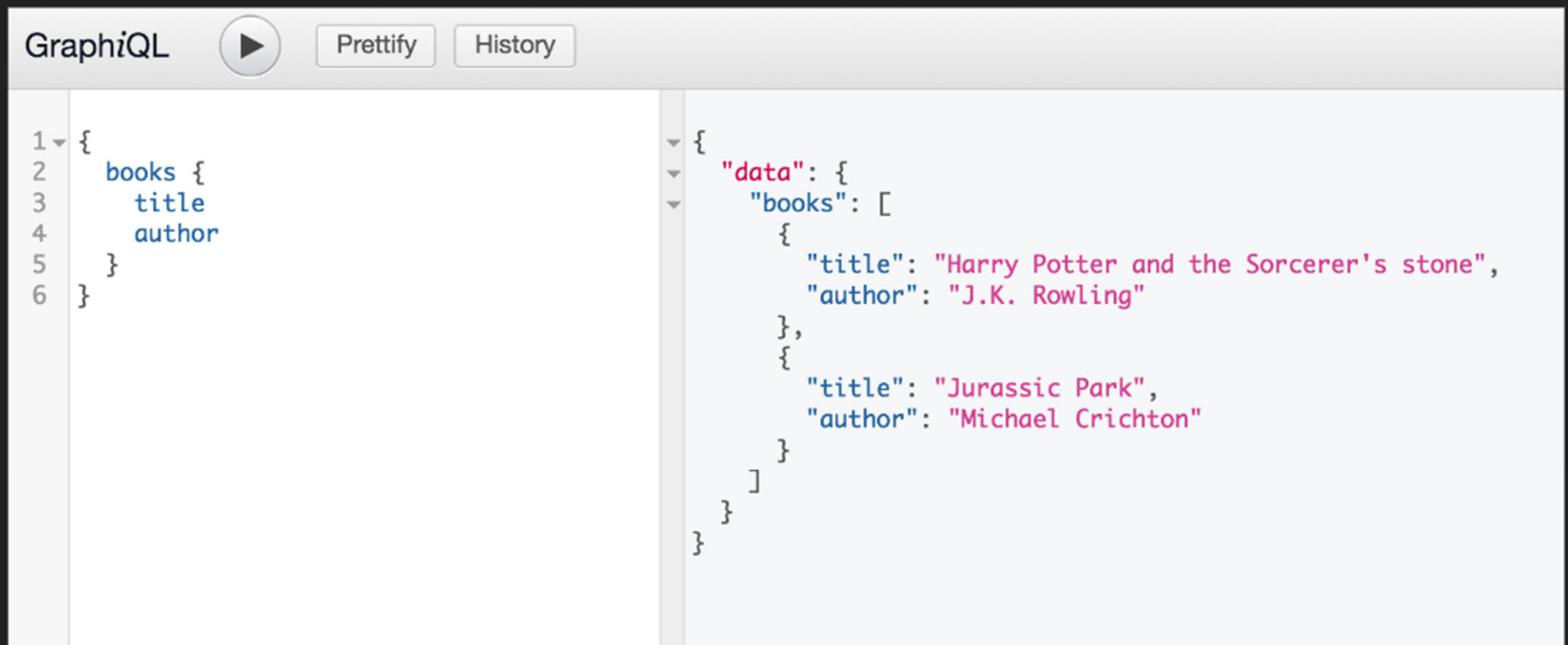
const schema = makeExecutableSchema({ typeDefs, resolvers });

const app = express();

app.use('/graphql', bodyParser.json(), graphqlExpress({ schema }));

app.listen(3000, () => {
  console.log('Go to http://localhost:3000/graphql to run queries!');
});
```

Fetch data with GraphiQL: Powerful GraphQL viewer, like Postman



Where is it used?



intuit.



Within IBM:

IBM Cloud Account microservice

SoftLayer VSI & bare metal provisioning microservice

Watson IoT (in development)

IBM Cloud IAM (in development)

Resources

GraphQL site

<https://graphql.org/learn/>

Apollo GraphQL site

<https://www.apollographql.com/>

How to GraphQL - great tutorial

<https://www.howtographql.com/>

IBM FED presentation on GraphQL

<https://jlengstorf.github.io/presentations/better-data-layer>